Frühjahr 2024
252-0027 – Einführung in die Programmierung

Departement Informatik
ETH Zürich

30. August 2024 – Schriftliche Prüfung

ID: 3335

Nachname:_____ Vorname:_____

Legi-Nummer:_____–_____–_____

Sie dürfen diese Prüfung oder die Aufgaben für die Programmierprüfung erst öffnen, nachdem die Aufsicht die Prüfung gestartet hat. **Wenn Sie diese Dokumente vorher öffnen, gilt dies als Täuschungsversuch.**

Mit Ihrer Unterschrift bestätigen Sie, dass Sie die hier aufgeführte Person sind, Sie die Hinweise zur Kenntnis genommen haben, Sie die Aufgaben selbständig gelöst haben, Sie Ihre eigene Lösung abgeben, Sie keine Kopie der Prüfung mitnehmen, und dass Sie alle technischen Probleme, gesundheitlichen Probleme (die Ihre Leistungen in dieser Prüfung beeinträchtigten) und etwaige störende äussere Einflüsse gemeldet haben bzw. wissen, dass Sie diese melden sollen.

Unterschrift:_____

Hinweise

1. Öffnen Sie diese Prüfung und die Aufgabenstellung für die Programmierprüfung erst, wenn die Aufsicht den Beginn der Prüfung bekannt gibt.
2. Schreiben Sie zuerst Ihren Namen und Ihre Legi-Nummer auf das Kontrolblatt dieser Prüfung. Überprüfen Sie, dass die Zahl auf dem Kontrolblatt mit der Zahl auf der ersten Seite der schriftlichen Prüfung übereinstimmt.
3. Vergessen Sie nicht Ihre Unterschrift nach dem Ende der schriftlichen Prüfung.
4. Der schriftliche Teil der Prüfung dauert 40 Minuten. Falls Sie sich durch irgendjemanden oder irgendetwas gestört fühlen, melden Sie dies sofort einer Aufsichtsperson.
5. Die schriftliche Prüfung hat 12 Seiten (inkl. Kontrolblatt). Vergewissern Sie sich, dass Ihr Exemplar vollständig ist.
6. In dieser Prüfung gibt es 40 Punkte. Benutzen Sie die Anzahl der Punkte als *Hinweis*, wie Sie Ihre Zeit einteilen können. Fehler bei der Bearbeitung einer Aufgabe haben keinen Einfluss auf die Punkte, die Sie für andere Aufgaben erhalten. Sie können die Aufgaben in beliebiger Reihenfolge lösen.
7. Lesen Sie die Aufgabenstellungen genau durch.
8. Tragen Sie Ihre Antwort(en) direkt in die Prüfungsbögen ein. Falls Sie mehr Platz brauchen, ist Ihre Antwort wahrscheinlich zu lang.
9. *Benutzen Sie einen Stift (blau oder schwarz), der nicht ausradiert werden kann.* Bitte schreiben Sie deutlich und leserlich! Wenn Sie etwas durchstreichen wollen, so machen Sie dies bitte klar und eindeutig.
10. Trennen Sie nicht die zusammengeheftete Prüfung. Mit losen Blättern riskieren Sie, ein Blatt zu verlieren.
11. Es ist wichtig, dass Ihre Antworten die Aufgaben klar und *unzweideutig* behandeln. Die Klarheit der Antworten beeinflusst Ihre Note. Eine fehlende Antwort wird als falsche Antwort bewertet.

    Wenn Sie Annahmen (über die in den Aufgaben aufgeführten hinaus) treffen, so geben Sie diese bitte an.
12. Wenn Sie zur Toilette müssen, melden Sie dies einer Aufsichtsperson durch Handzeichen. Es darf zur gleichen Zeit immer nur eine Person zur Toilette.
13. Wir beantworten keine inhaltlichen Fragen während der Prüfung. Kommentare zur Aufgabe schreiben Sie bitte auf das Aufgabenblatt.
14. Wenn die Aufsicht diesen Teil der Prüfung beendet, schliessen Sie bitte die Prüfung und schreiben nicht mehr in die Prüfung. **Weiterarbeiten über die erlaubte Zeit gilt als Täuschungsversuch.** Bitte unterschreiben Sie die Prüfung auf der Vorderseite und legen Sie die Prüfung mit Ihrer Legitimationskarte ("Legi") gut sichtbar auf Ihren Arbeitstisch.
15. Wir sammeln die Prüfung ein. Wichtig: stellen Sie unbedingt selbst sicher, dass Ihre Prüfung von einer Aufsichtsperson eingezogen wird. Stecken Sie keine Prüfung (auch keine leere) ein.

ID: 3335

Aufgabe	Wert	Punkte		Aufgabe	Wert	Punkte	
1	4			5	8		
2	5			6	5		
3	4			7	6		
4	8						
Σ				Übertrag	−−		
				Total	40		

Aufgabe 1 (4)

Gegeben sei eine Methode `main` in einer Java-Klasse.

```
public static void main(String[] args) {
    /* body */
}
```

Die folgenden Anweisungen sollen als "Body" (Rumpf) anstelle des Kommentars `/* body */` eingefügt werden. Geben Sie für jede Anweisung an, was für eine Ausgabe erzeugt wird – entweder was gedruckt wird oder ob ein Laufzeitfehler auftritt (schreiben Sie "Laufzeitfehler" oder "Exception") oder ob der Compiler einen Fehler feststellt (schreiben Sie "Compile-Fehler" oder "Error"). Achten Sie auf die korrekte Formatierung der verschiedenen Typen, also z.B. 7.0 statt 7 für eine reelle Zahl (`double`).

1. `System.out.println( "a".equals("A") || (5++ == 7) );`

   _____

2. `System.out.println(  8 / 6 * (double) (6 / 4) == (12 / 8.0) || (12 % (6 / 8) == 0) );`

   _____

3. `System.out.println( (1 % 2) + 3 * ( (double) 9 / 6) + ("5" + 9 % 5 + 4 / 6) );`

   _____

Aufgabe 2 (5)

Was druckt die Methode `main` der folgenden Klasse `Beispiel`?

```
class Beispiel {
    public static void main(String[] args) {

        int[] a = new int[6];

        int j = 0;

        for (int i=a.length-1; i>0; i-- ) {
            a[i] = 1 +  Math.min(i,j);
            j++;
        }

       int sum = 0;

       for (int k : a) {
           sum += a[k];
       }

        System.out.println("sum = " + sum);    // sum = ____________________

    }
}
```

Aufgabe 3 (4)

Gegeben sei in Abbildung 1 die EBNF-Beschreibung von *expression*. Für die EBNF-Beschreibung von *identifier* gelten die Regeln für Bezeichner in Java. Die EBNF-Beschreibung von *expression* unterscheidet sich aber sonst von den in Java zulässigen Ausdrücken.

shift_op ⇐ `«` | `»`
operator ⇐ `+` | `-` | `*` | `/`
factor ⇐ *factor operator identifier* | *identifier*
term1 ⇐ *factor* `*` *identifier* | *identifier*
term2 ⇐ `(` *factor operator identifier* `)` | *identifier*
expression ⇐ *term1 shift_op term2* | *term2*

Abbildung 1: **EBNF-Beschreibung** von *expression*

Geben Sie für jeden folgenden Ausdruck an, ob er nach der EBNF-Beschreibung von *expression* in Abbildung 1 gültig ist. (Tipp: alle Bezeichner und Werte in diesen Ausdrücken (d.h., *identifier* in der Abbildung 1) sind korrekt.)

Ausdruck	Gültig	Ungültig	Ausdruck	Gültig	Ungültig
`a « ( 5 + b )`			`b « ( x + y * b )`		
`v » ( u - u )`			`r « s * ( t + t )`		
`k « ( l * ( m + n ) )`			`( a * b ) » c`		

Aufgabe 6 (5)

Gegeben ist die Methode `compute(int n, int d)`, die für eine ganze Zahl $n \geq 1$ den Ausdruck

$$\frac{n \cdot (n-1) \cdot d}{2}$$

berechnet. (Sie können davon ausgehen, dass die Parameter eines Methodenaufrufs sicherstellen, dass alle Berechnungen korrekt und ohne Overflow oder Underflow ausgeführt werden.)

Was ist die Invariante für den Loop in dieser Methode?

```
   static int compute(int n, int d) {
 // Precondition: n >= 1
   int res = 0;
   int k = 1;
 // Loop Invariante:  Bitte geben Sie die Invariante unten an
   while (k <= n)   {
      res = res + (k-1) * d;
      k = k + 1;
   }
// Postcondition: res == (n·(n−1)·d)/2
      return res;
}
```

Geben Sie die Loop Invariante an. (Sie können, wenn Sie es für nötig halten, die Precondition oben erweitern.)

Loop Invariante: _____

_____

Aufgabe 7 (6)

Bitte geben Sie für die folgenden Java-Programmsegmente die schwächste Vorbedingung (weakest precondition) WP an. Bitte geben Sie die Precondition als Java-Expression an. Alle Variablen sind vom Typ `int` und initialisiert; es gibt keinen Over-/Underflow.

1. `WP: {` `}`

   ```
   f = e * 2;
   g = f + 3;

   Q: { g >= 12 }
   ```

2. `WP: {` `}`

   ```
   y = x * x;
   z = (x+1) * (x+1);
   w = y + z;

   Q: { w > 0 }
   ```

3. `WP: {` `}`

   ```
   r = s + 2;
   if (r < 0) {
     t  = s * s;
   } else {
     t = r - 4;
   }

   Q: { t > 0 }
   ```

Wir wünschen Ihnen alles Gute für das nächste Semester. Ihr "Einführung in die Programmierung"-Team.

Aufgabe	Wert	Punkte		Aufgabe	Wert	Punkte	
1	4			5	8		
2	5			6	5		
3	4			7	6		
4	8						
Σ				Übertrag	−−		
				Total	40		